```
# ==== SETUP: install and import libraries ====
# Run this cell once whenever you (re)connect to the runtime.

# 1. Install any libraries
!pip install pandas matplotlib seaborn --quiet

# 2. Import all the core libraries you'll use
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 3. Global display options (optional)
pd.set_option("display.max_rows", 10)
sns.set(style="whitegrid")

print("Setup complete")
```

```
Setup complete
```

New Section

```
# Connect this notebook to your Google Drive
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
from google.colab import files
import pandas as pd
uploaded = files.upload()           # you chose "student.csv or GDP nominal cpita file from
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving GDP (nominal) per Capita.csv to GDP (nominal) per Capita.csv
```

```
full_path = '/content/student.csv'  # match the name shown after upload.
```

```
df = pd.read_csv('/content/GDP (nominal) per Capita.csv')            # file that appears on
df.head()
```

	Unnamed: 0	Country/Territory	UN_Region	IMF_Estimate	IMF_Year	WorldBank_Estimate	WorldB
0	1	Monaco	Europe	0	0	234316	
1	2	Liechtenstein	Europe	0	0	157755	
2	3	Luxembourg	Europe	132372	2023	133590	
3	4	Ireland	Europe	114581	2023	100172	
4	5	Bermuda	Americas	0	0	114090	

```
print("Dataset loaded: Filename.csv")
print(f"Shape: {df.shape}")
```

```
Dataset loaded: Filename.csv
Shape: (223, 9)
```

```
df['Country/Territory'] = df['Country/Territory'].str.strip()
```

```
df.head()
```

	Unnamed: 0	Country/Territory	UN_Region	IMF_Estimate	IMF_Year	WorldBank_Estimate	WorldB
0	1	Monaco	Europe	0	0	234316	
1	2	Liechtenstein	Europe	0	0	157755	
2	3	Luxembourg	Europe	132372	2023	133590	
3	4	Ireland	Europe	114581	2023	100172	
4	5	Bermuda	Americas	0	0	114090	

```
df['Country/Territory'] = df['Country/Territory'].str.strip()
```

```
df.head()
```

	Unnamed: 0	Country/Territory	UN_Region	IMF_Estimate	IMF_Year	WorldBank_Estimate	WorldB
0	1	Monaco	Europe	0	0	234316	
1	2	Liechtenstein	Europe	0	0	157755	
2	3	Luxembourg	Europe	132372	2023	133590	
3	4	Ireland	Europe	114581	2023	100172	
4	5	Bermuda	Americas	0	0	114090	

```
df['column'].str.strip():
```

```
  File "/tmp/ipykernel_8054/1976891433.py", line 1
    df['column'].str.strip():
                             ^
SyntaxError: invalid syntax
```

```
df['column'] = df['column'].astype(str).str.strip()
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in
pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in
pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'column'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
                                    2 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3810             ):
   3811                 raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
   3813         except TypeError:
   3814             # If we have a listlike key, _check_indexing_error will raise
```

```
df['Country/Territory'] = df['Country/Territory'].astype(str).str.strip()
```

```
df.head()
```

	Unnamed: 0	Country/Territory	UN_Region	IMF_Estimate	IMF_Year	WorldBank_Estimate	WorldB
0	1	Monaco	Europe	0	0	234316	
1	2	Liechtenstein	Europe	0	0	157755	
2	3	Luxembourg	Europe	132372	2023	133590	
3	4	Ireland	Europe	114581	2023	100172	
4	5	Bermuda	Americas	0	0	114090	

```
pd.to_datetime(df['IMF_Year']):
```

```
  File "/tmp/ipykernel_8054/2347006509.py", line 1
    pd.to_datetime(df['IMF_Year']):
                                   ^
SyntaxError: invalid syntax
```

```
pd.to_datetime(df['IMF_Year'])
```

	IMF_Year
0	1970-01-01 00:00:00.000000000
1	1970-01-01 00:00:00.000000000
2	1970-01-01 00:00:00.000002023
3	1970-01-01 00:00:00.000002023
4	1970-01-01 00:00:00.000000000
...	...
218	1970-01-01 00:00:00.000002023
219	1970-01-01 00:00:00.000002023
220	1970-01-01 00:00:00.000002023
221	1970-01-01 00:00:00.000002020
222	1970-01-01 00:00:00.000002023

223 rows × 1 columns

dtype: datetime64[ns]

```
df.head()
```

	Unnamed: 0	Country/Territory	UN_Region	IMF_Estimate	IMF_Year	WorldBank_Estimate	WorldB
0	1	Monaco	Europe	0	0	234316	
1	2	Liechtenstein	Europe	0	0	157755	
2	3	Luxembourg	Europe	132372	2023	133590	
3	4	Ireland	Europe	114581	2023	100172	
4	5	Bermuda	Americas	0	0	114090	

```
print(df[IMF_Year].head())
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_8054/2798119026.py in <cell line: 0>()
----> 1 print(df[IMF_Year].head())

NameError: name 'IMF_Year' is not defined
```

```
# c) Convert 'WorldBank_Estimate' to integer.
df['WorldBank_Estimate'] = pd.to_numeric(df['WorldBank_Estimate'], errors='coerce').fillna(0
```

```
print(df.dtypes)
```

```
Unnamed: 0             int64
Country/Territory     object
UN_Region             object
IMF_Estimate           int64
IMF_Year               int64
WorldBank_Estimate     int64
WorldBank_Year         int64
UN_Estimate            int64
UN_Year               object
dtype: object
```

```
Start coding or generate with AI.
```

```
col = 'IMF_Estimate'

Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df[col] < lower) | (df[col] > upper)]
```

```
outliers.head()
```

	Unnamed: 0	Country/Territory	UN_Region	IMF_Estimate	IMF_Year	WorldBank_Estimate	WorldB
2	3	Luxembourg	Europe	132372	2023	133590	
3	4	Ireland	Europe	114581	2023	100172	
5	6	Norway	Europe	101103	2023	89154	
6	7	Switzerland	Europe	98767	2023	91992	
7	8	Singapore	Asia	91100	2023	72794	

```
df.shape
```

```
(223, 9)
```

```
# 3. Print outliers shape
print("Outliers shape:", outliers.shape)
```

```
Outliers shape: (23, 9)
```

```
# b) Convert 'IMF_Year' to datetime
df['IMF_Year'] = pd.to_datetime(df['IMF_Year'], format='%Y')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_8054/3127212490.py in <cell line: 0>()
      1 # b) Convert 'IMF_Year' to datetime
----> 2 df['IMF_Year'] = pd.to_datetime(df['IMF_Year'], format='%Y')

 3 frames 

/usr/local/lib/python3.12/dist-packages/pandas/core/tools/datetimes.py in
_array_strptime_with_fallback(arg, name, utc, fmt, exact, errors)
    465     Call array_strptime, with fallback behavior depending on 'errors'.
    466     """
--> 467     result, tz_out = array_strptime(arg, fmt, exact=exact, errors=errors, utc=utc)
    468     if tz_out is not None:
    469         unit = np.datetime_data(result.dtype)[0]

strptime.pyx in pandas._libs.tslibs.strptime.array_strptime()

strptime.pyx in pandas._libs.tslibs.strptime.array_strptime()

strptime.pyx in pandas._libs.tslibs.strptime._parse_with_format()

ValueError: time data "0" doesn't match format "%Y", at position 0. You might want to try:
    - passing `format` if your strings have a consistent format;
    - passing `format='ISO8601'` if your strings are all ISO8601 but not necessarily in
exactly the same format;
    - passing `format='mixed'`, and the format will be inferred for each element
individually. You might want to use `dayfirst` alongside this.
```

```
df['IMF Year'] = pd.to_datetime(df['IMF Year'], format='%Y', errors='coerce')
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in
pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in
pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'IMF Year'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
                                  2 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3810             ):
   3811                 raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
   3813         except TypeError:
   3814             # If we have a listlike key, _check_indexing_error will raise
```

```
df['IMF_Year'] = pd.to_datetime(
    df['IMF_Year'],
    format='%Y',
    errors='coerce'
)
```

```
df.head()
```

	Unnamed: 0	Country/Territory	UN_Region	IMF_Estimate	IMF_Year	WorldBank_Estimate	WorldB
0	1	Monaco	Europe	0	NaT	234316	
1	2	Liechtenstein	Europe	0	NaT	157755	
2	3	Luxembourg	Europe	132372	2023-01-01	133590	
3	4	Ireland	Europe	114581	2023-01-01	100172	
4	5	Bermuda	Americas	0	NaT	114090	

```
print(df['IMF_Year'].head())
```

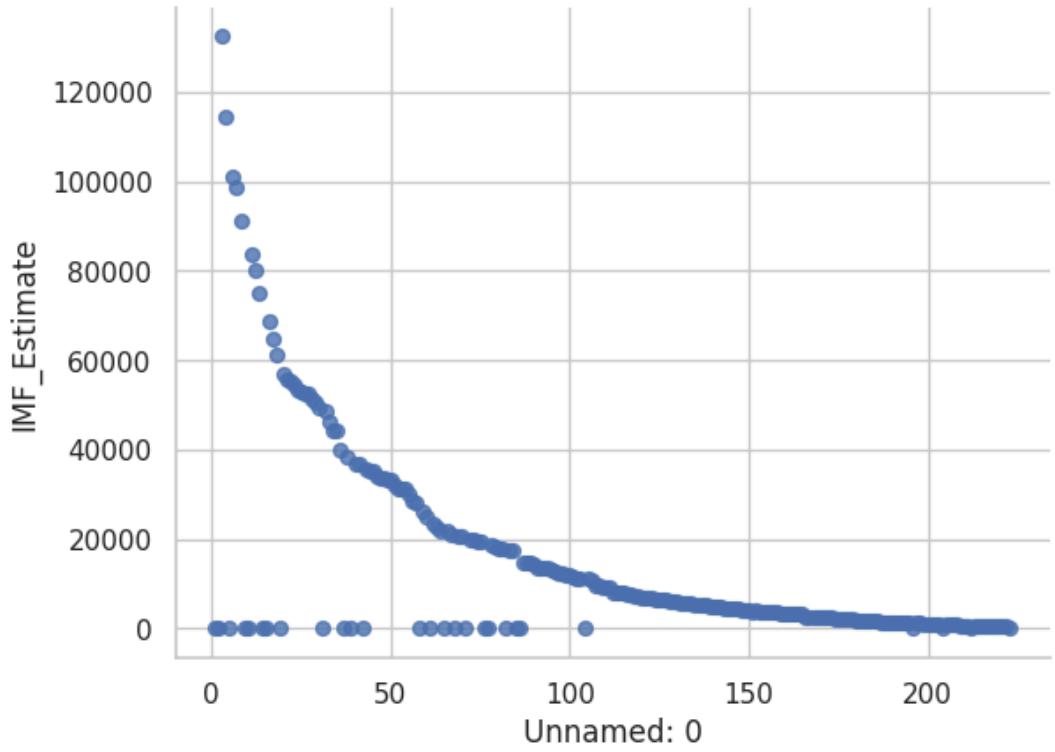
```
0          NaT
1          NaT
```

```
2   2023-01-01
3   2023-01-01
4          NaT
Name: IMF_Year, dtype: datetime64[ns]
```

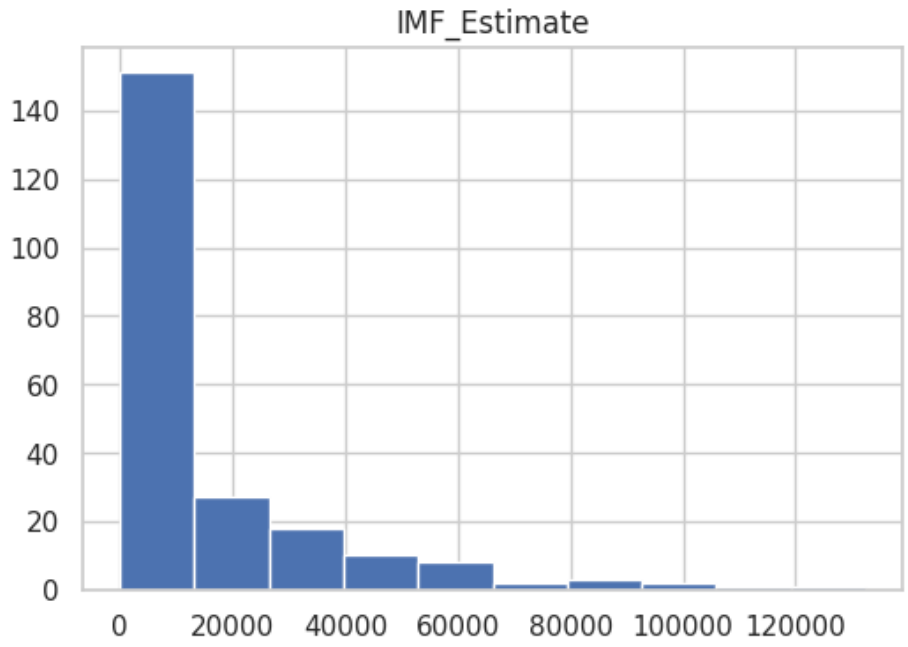
```
print(df['IMF_Year'].dtype)
```

```
datetime64[ns]
```

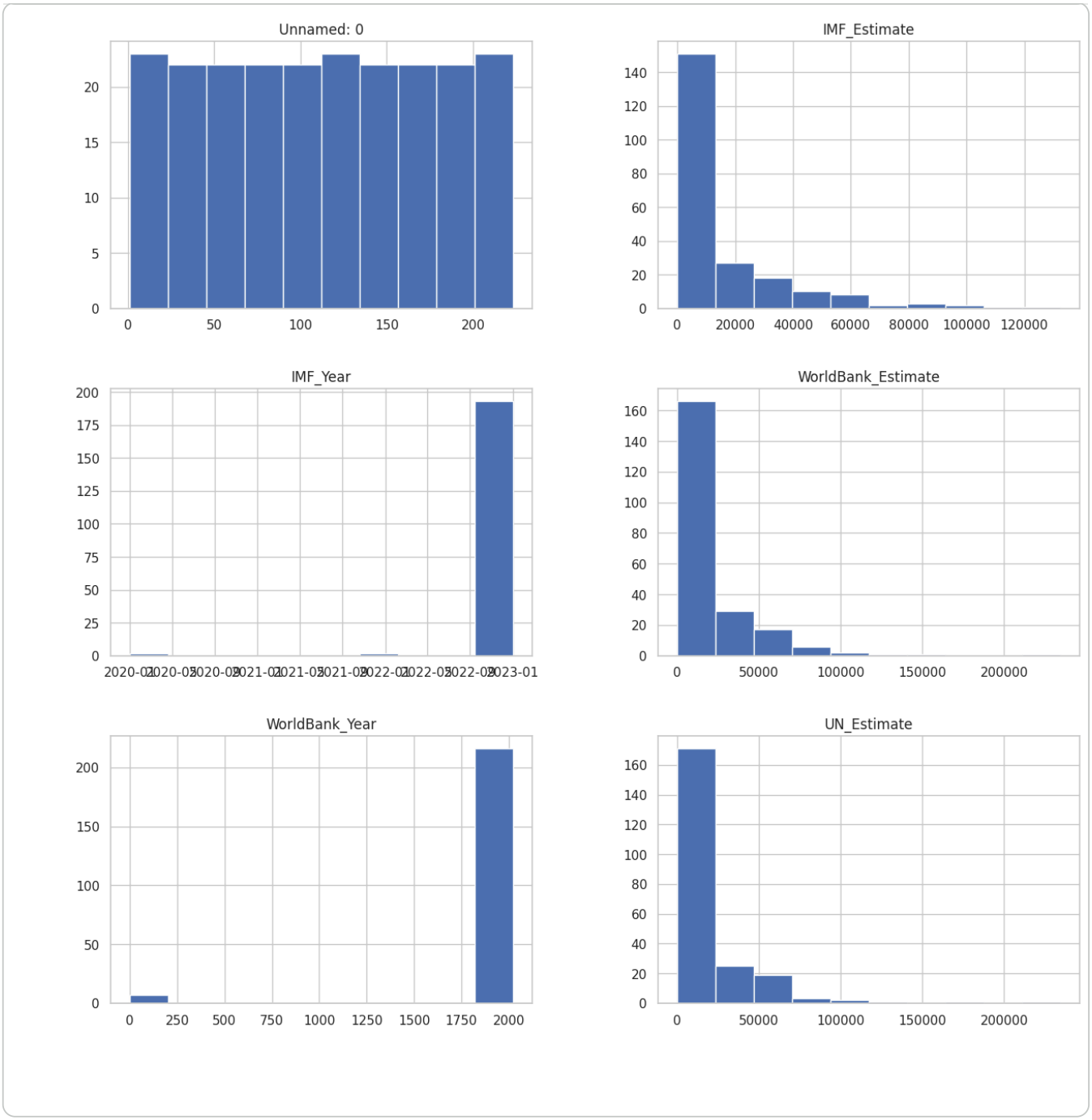
```
df.plot(kind='scatter', x='Unnamed: 0', y='IMF_Estimate', s=32, alpha=.8)
plt.gca().spines[['top', 'right',]].set_visible(False)
```




```
df.hist(column='IMF_Estimate', figsize=(6, 4))
plt.show()
```




```
df.hist(figsize=(15, 15))
plt.show()
```




```
!pip install seaborn
```

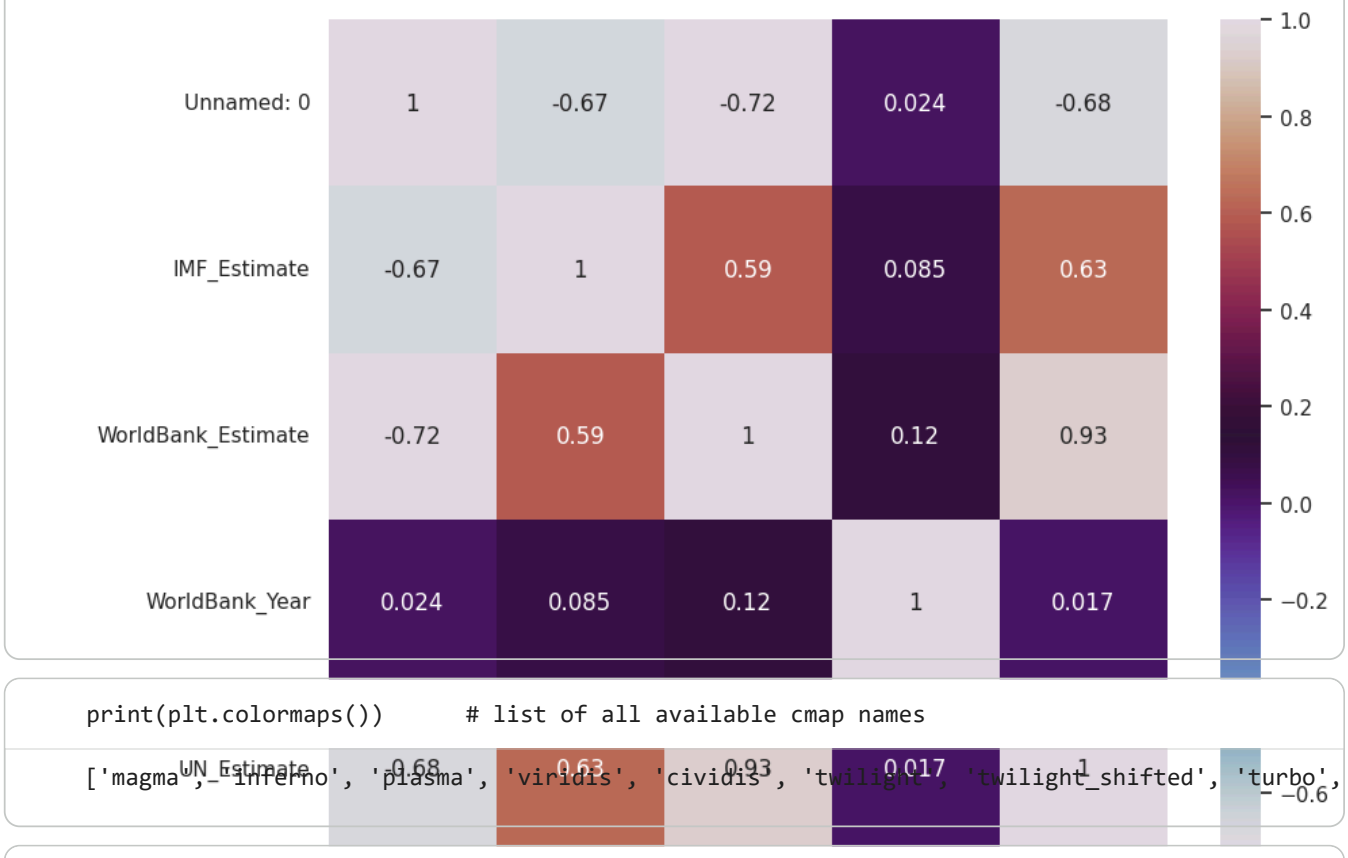
```
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from s
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from mat
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from pyth
```

```
import seaborn as sns
#Import the seaborn module.
```

```
# Select only the numerical features for correlation analysis
numerical_features = df.select_dtypes(include=['number'])
```

```
# Calculate the correlation matrix
corr = numerical_features.corr()
```

```
# Plot the heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, cmap='twilight')
plt.show()
```

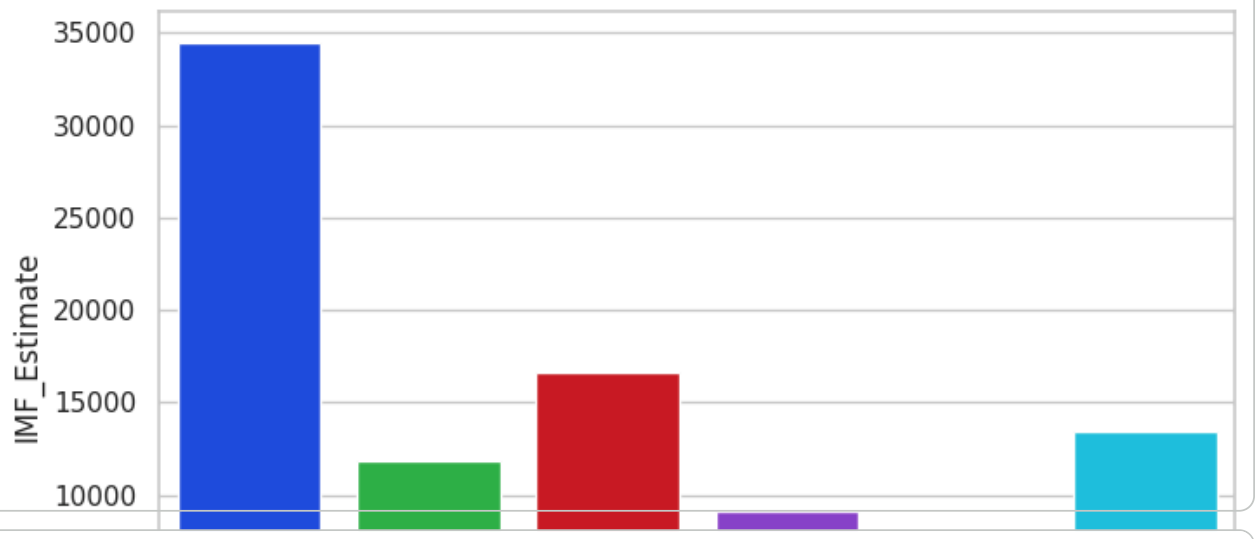



```
print(plt.colormaps())      # list of all available cmap names
```

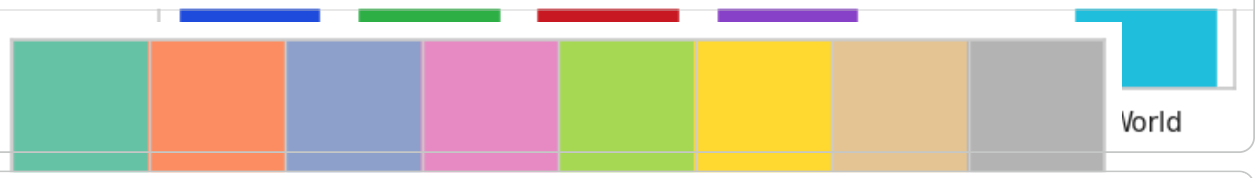
```
['magma', 'inferno', 'plasma', 'viridis', 'cividis', 'twilight', 'twilight_shifted', 'turbo',
```

```
fig = plt.figure(figsize=(8, 5)) #Now plt is defined and can be used.
sns.barplot(x="UN_Region", y="IMF_Estimate", data=df, errorbar=None, palette="bright6")
plt.show()
```

```
/tmp/ipykernel_8054/1567482934.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assig

  sns.barplot(x="UN_Region", y="IMF_Estimate", data=df, errorbar=None, palette="bright6")
```



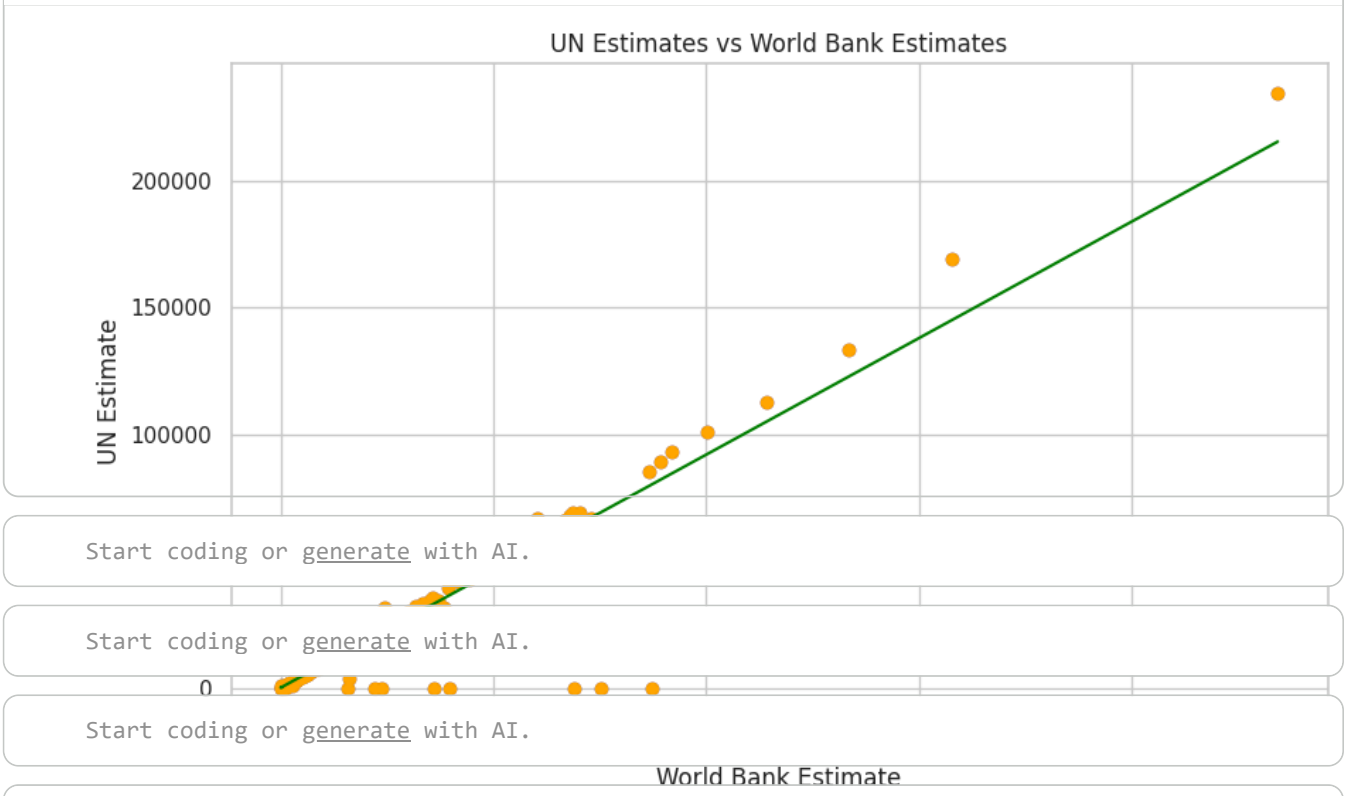

```
sns.palplot(sns.color_palette("Set2")) # try "Set2", "Set3", "pastel", "bright", etc. plt.sh
```




```
print(sns.palettes.SEABORN_PALETTES.keys())
```

```
dict_keys(['deep', 'deep6', 'muted', 'muted6', 'pastel', 'pastel6', 'bright', 'bright6', 'dar
```

```
#scatter plot

plt.figure(figsize=(10, 6))
plt.scatter(df['WorldBank_Estimate'], df['UN_Estimate'], color='blue', alpha=0.6)

plt.title('UN Estimates vs World Bank Estimates')
plt.xlabel('World Bank Estimate')
plt.ylabel('UN Estimate')

plt.grid(True)

import numpy as np

# Calculate the trend line and a colour:
m, b = np.polyfit(df['WorldBank_Estimate'], df['UN_Estimate'], 1)  # Linear fit
plt.plot(df['WorldBank_Estimate'], m * df['WorldBank_Estimate'] + b, color='green', linestyl


#Then add this line of code to change the colour of dots
plt.scatter(df['WorldBank_Estimate'], df['UN_Estimate'], color='orange') # colour of dots


plt.show()
```



Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
from google.colab import files
uploaded = files.upload()

#Choose file > GDP nominal per capita.csv
```

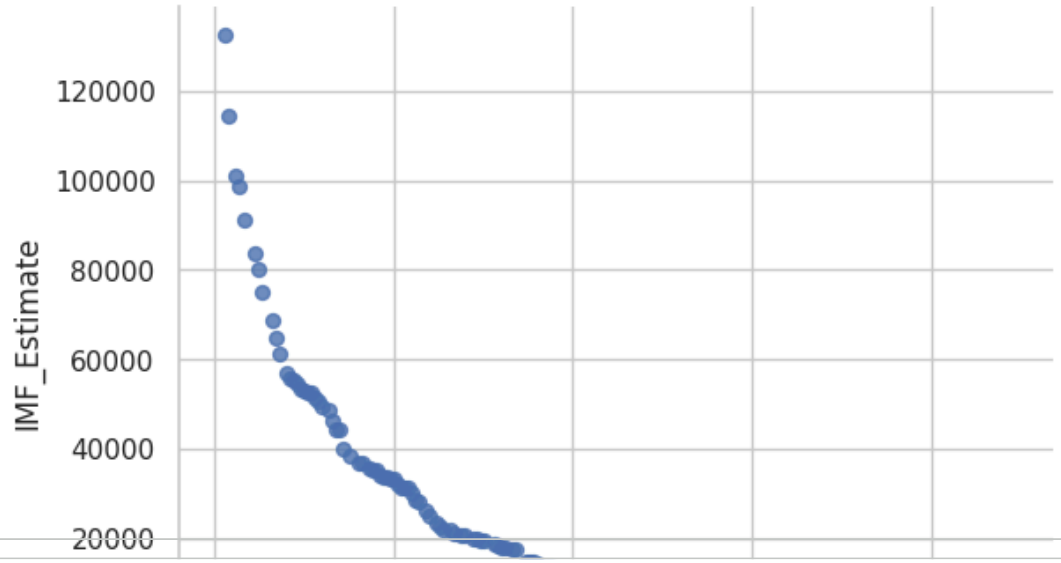
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving GDP (nominal) per Capita.csv to GDP (nominal) per Capita (1).csv
```

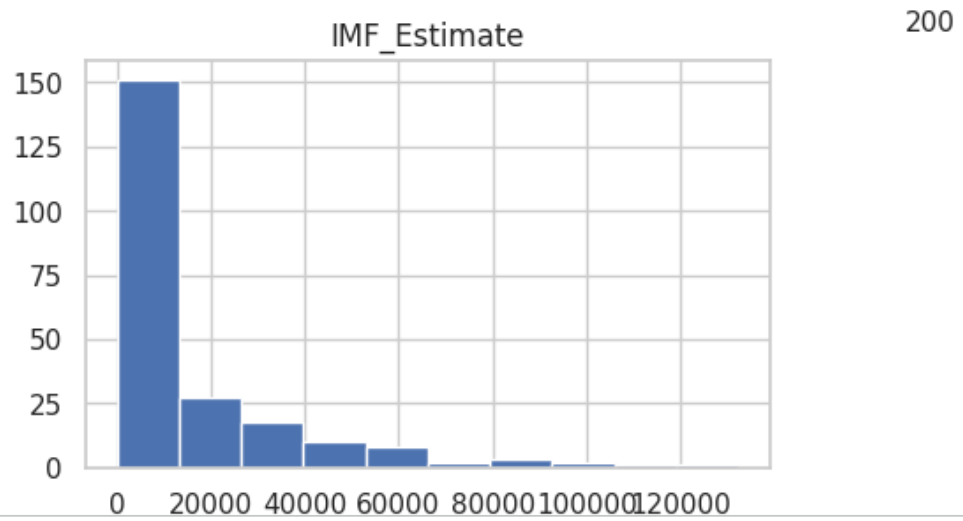
```
import pandas as pd
df = pd.read_csv('GDP (nominal) per Capita.csv')
```

```
from matplotlib import pyplot as plt
```

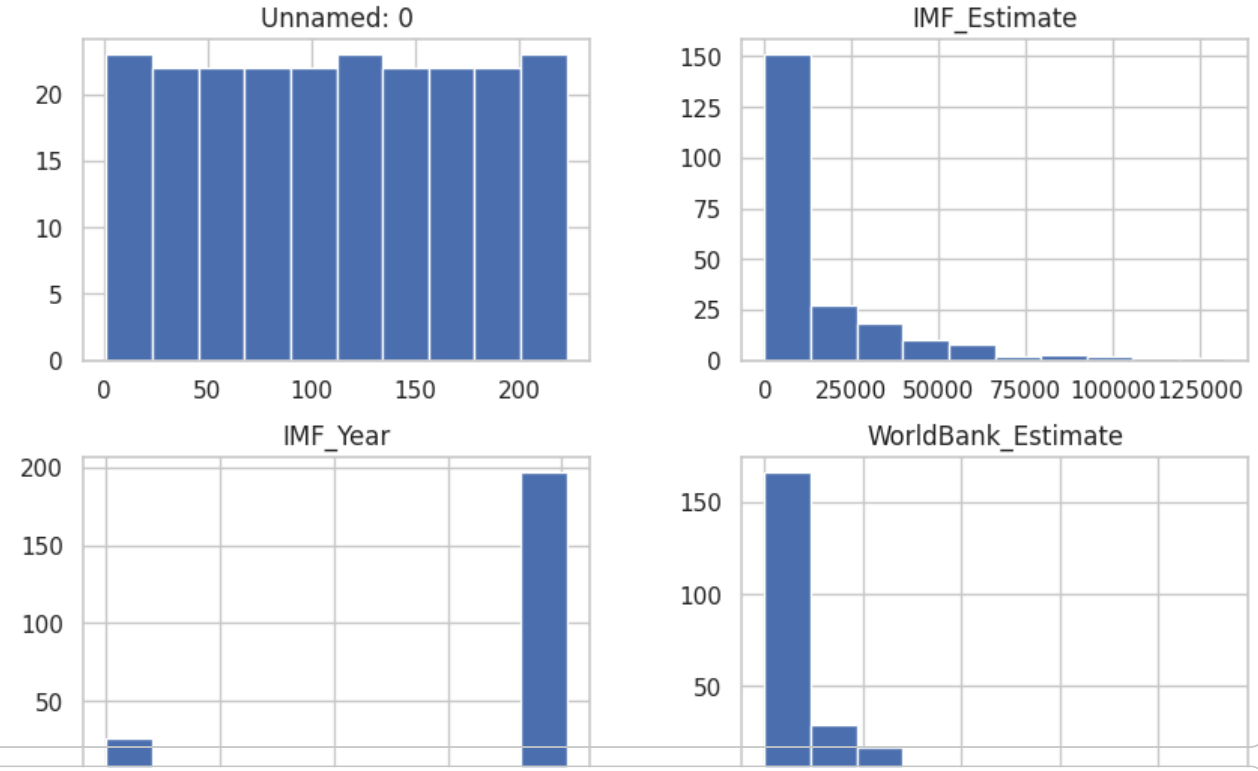
```
df.plot(kind='scatter', x='Unnamed: 0', y='IMF_Estimate', s=32, alpha=.8)
plt.gca().spines[['top', 'right',]].set_visible(False)
```




```
df.hist(column='IMF_Estimate',figsize=(5,3))
plt.show()
```




```
df.hist(figsize=(10, 10))
plt.show()
```




```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
!pip install seaborn
```

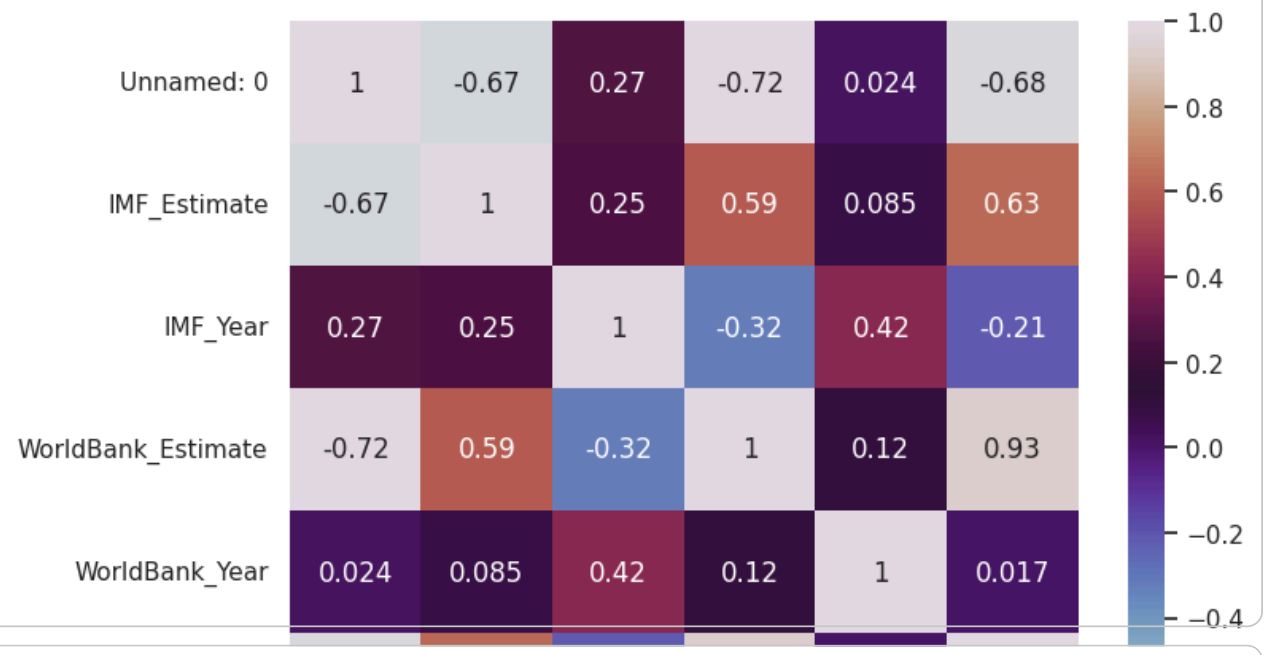
```
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from s
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from mat
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from pyth
```

```
import seaborn as sns
#Import the seaborn module.
```

```
# Select only the numerical features for correlation analysis
numerical_features = df.select_dtypes(include=['number'])
```

```
# Calculate the correlation matrix
corr = numerical_features.corr()
```

```
#plot a heatmap
plt.figure(figsize=(8,6))
sns.heatmap(corr,annot=True,cmap='twilight')
plt.show()
```

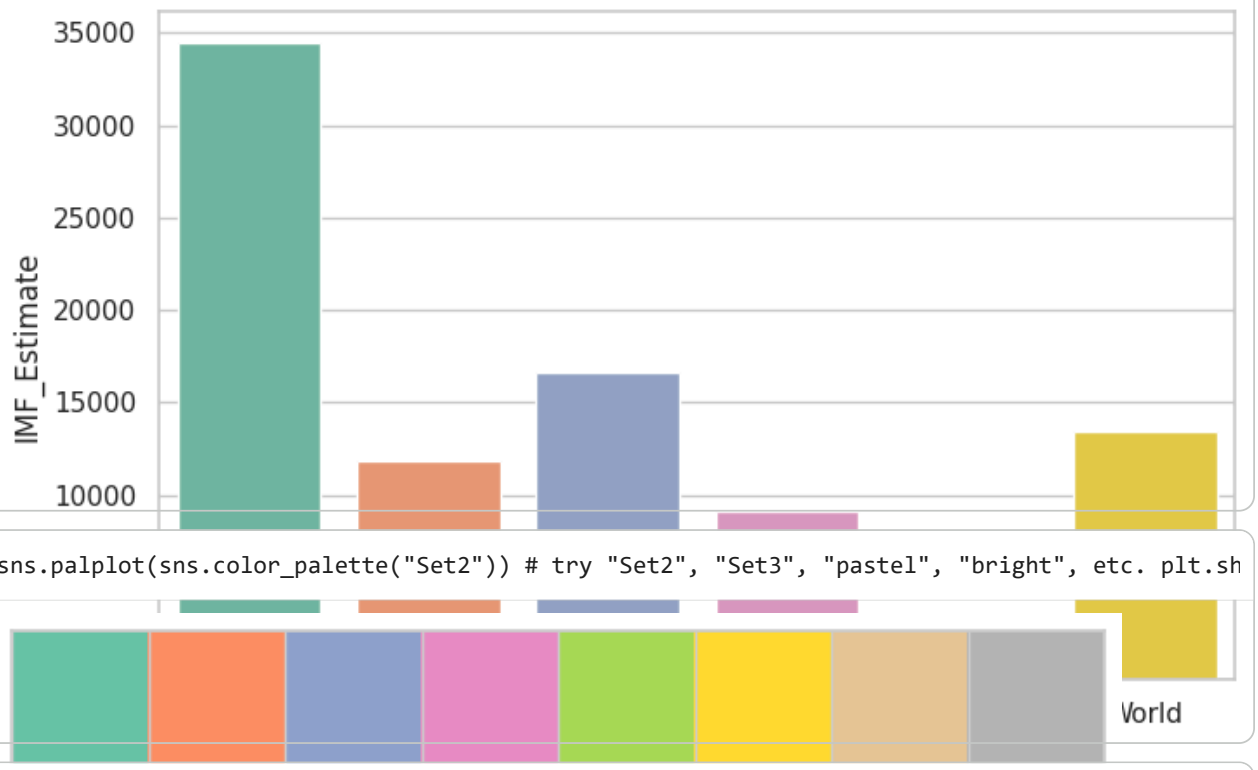



```
print(plt.colormaps())
```

```
['magma', 'inferno', 'plasma', 'viridis', 'cividis', 'twilight', 'twilight_shifted', 'turbo',
```

```
fig = plt.figure(figsize=(8, 5)) #Now plt is defined and can be used.
fig = plt.figure(figsize=(8, 5))  #Now plt is defined and can be used.
sns.barplot(x="UN_Region", y="IMF_Estimate", data=df, errorbar=None, palette="Set2")
plt.show()
```

```
/tmp/ipykernel_8054/3576313189.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assig

  sns.barplot(x="UN_Region", y="IMF_Estimate", data=df, errorbar=None, palette="Set2")
<Figure size 800x500 with 0 Axes>
```



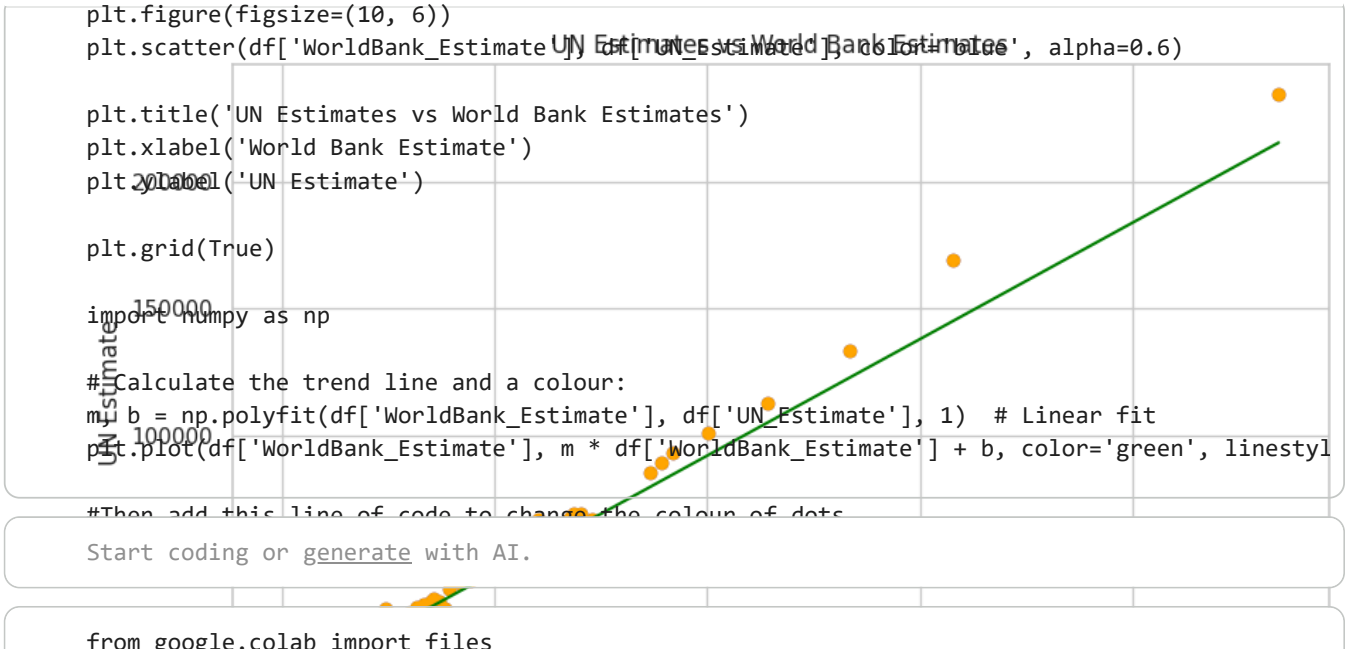

```
sns.palplot(sns.color_palette("Set2")) # try "Set2", "Set3", "pastel", "bright", etc. plt.sh
```



```
print(sns.palettes.SEABORN_PALETTES.keys()) #previously I gave you colour palette for Cmaps,
```

```
dict_keys(['deep', 'deep6', 'muted', 'muted6', 'pastel', 'pastel6', 'bright', 'bright6', 'dar
```

```
plt.figure(figsize=(10, 6))
plt.scatter(df['WorldBank_Estimate'], df['UN_Estimate'], color='blue', alpha=0.6)

plt.title('UN Estimates vs World Bank Estimates')
plt.xlabel('World Bank Estimate')
plt.ylabel('UN Estimate')

plt.grid(True)

import numpy as np

#Calculate the trend line and a colour:
m, b = np.polyfit(df['WorldBank_Estimate'], df['UN_Estimate'], 1)  # Linear fit
plt.plot(df['WorldBank_Estimate'], m * df['WorldBank_Estimate'] + b, color='green', linestyl

#Then add this line of code to change the colour of dots
```



Start coding or generate with AI.

```
from google.colab import files
```